VerifyWise

How VerifyWise evaluates LLM systems

A reference for risk, compliance and AI governance officers describing how the VerifyWise LLM Evals module measures, documents and reports the quality, safety and behavior of large language model systems used in chatbots, retrieval-augmented generation and agentic workflows.

Prepared by	Audience	Date	Version
VerifyWise AI governance team	Risk, compliance & AI governance officers	May 22, 2026	1.0

Executive summary

This document explains how the VerifyWise LLM Evals module evaluates language model systems. It is written for risk and compliance teams who need to satisfy NIST AI RMF, ISO/IEC 42001 and EU AI Act expectations for LLM-based features in production.

The module rests on three commitments that a regulated organization should expect from an LLM evaluation tool:

1. **Evaluations are repeatable.** Each experiment captures its configuration (the model, the dataset, the judge, the metrics, the thresholds) and stores it alongside the per-row results. The same experiment can be re-run later against a different model or a new release of the same model and compared row by row.
2. **Each metric is documented.** Every metric used to score an LLM output is defined in the report: what it measures, whether it requires ground truth, whether a higher or lower score is better, and the threshold that determines pass or fail. There are no opaque composite scores.
3. **The output is a regulator-ready artifact.** Each experiment produces a dated PDF that documents the configuration, the metrics, the per-prompt results and the known limitations, suitable for inclusion in an audit response or a model inventory record.

The remainder of this document walks through how the module is organized, what it measures, how an experiment is configured and produced, how findings feed the AI risk register, and how each output maps to NIST AI RMF, ISO/IEC 42001 and the EU AI Act. Built-in datasets and supported providers are listed as appendices.

1. The LLM Evals module

The LLM Evals module is one component of the VerifyWise AI governance platform. It runs inside the **Evals workspace** and integrates with the platform's model inventory, risk register, policy manager and use case register.

How work is organized

The module follows a project-then-experiment structure.

- **Projects** group experiments by use case. Each project is tagged with one of three use cases: chatbot, retrieval-augmented generation (RAG), or agent. The use case determines which metrics the module makes available.
- **Experiments** are individual runs inside a project. Each experiment is a complete configuration: the model under evaluation, the dataset, the judge model, the metrics and the thresholds.
- **Runs** produce per-row results stored against the experiment, with each prompt, response, score and pass/fail status recorded individually.

A typical workflow

1. The AI use case (for example, a customer support chatbot) is recorded in the VerifyWise model inventory and linked to the applicable regulatory frameworks: NIST AI RMF, ISO/IEC 42001 and any sector-specific rules.
2. Before deployment, and on a recurring cadence (typically after each prompt or model change), the system owner runs experiments against a representative dataset.
3. Each experiment produces a dated PDF and a per-row results table. Both are stored in the Evidence Hub and attached to the model record.
4. Failed thresholds prompt a remediation entry in the AI risk register, with an assigned owner and a due date. The handoff is currently manual; see Section 06.

2. What the module measures

The module evaluates an LLM system along three groups of metrics. The set of metrics offered to the user depends on the project's use case.

Universal metrics (every use case)

These metrics apply to any LLM output and are LLM-judged. They are computed by passing each (input, output, optional expected output) triplet to a judge model that returns a numeric score.

- **Answer relevancy** — how well the response addresses the question asked
- **Correctness** — factual accuracy compared to a known correct answer (requires ground truth)
- **Completeness** — how thoroughly the response covers the expected content (requires ground truth)
- **Hallucination** — presence of unsupported claims; lower score is better
- **Instruction following** — whether the response respects the instructions in the prompt
- **Toxicity** — presence of toxic, abusive or unsafe content; lower score is better
- **Bias** — presence of biased framing or stereotypes; lower score is better

Four of these (hallucination, toxicity, bias, conversation safety) are inverted: a low numeric score corresponds to a good outcome. The threshold convention is documented in the PDF so the reader does not have to interpret the direction.

RAG-specific metrics

Added automatically when the project use case is RAG. The dataset must include a **context** array for each prompt.

- **Faithfulness** — how well the answer is grounded in the retrieved context
- **Context relevancy** — how relevant the retrieved context is to the question
- **Context precision** — proportion of retrieved context that is relevant
- **Context recall** — proportion of relevant context that was retrieved

Together these four metrics characterize both the retrieval step and the generation step. A drop in faithfulness with stable context relevancy usually indicates a generation problem; a drop in context recall usually indicates a retrieval problem.

Agent-specific metrics

Added automatically when the project use case is agent. Designed for systems that plan, call tools and execute multi-step tasks.

- **Plan quality** — coherence and reasonableness of the planned steps
- **Plan adherence** — whether the agent followed its own plan
- **Tool correctness** — whether the right tool was selected for each step
- **Argument correctness** — whether the arguments passed to each tool were valid
- **Task completion** — whether the task was completed end to end
- **Step efficiency** — number of steps used relative to the minimum required

Multi-turn chatbot metrics

Added when the dataset is multi-turn (each row contains a conversation rather than a single prompt).

- **Turn relevancy** — each response stays relevant to the current turn
- **Knowledge retention** — the system remembers earlier turns
- **Conversation coherence** — the conversation flows logically
- **Conversation helpfulness** — the conversation makes progress toward the user's goal
- **Conversation safety** — safety is maintained across all turns; lower is better

Custom scorers (user-defined)

In addition to the built-in metrics, users can define custom scorers. A custom scorer is a judge prompt template (with **{{input}}**, **{{output}}** and **{{expected}}** placeholders) plus a set of labeled outcomes (for example, PASS and FAIL) with numeric scores. Custom scorers are stored against the organization and can be reused across experiments.

Custom scorers support structured JSON responses and tolerate chain-of-thought traces (for example, the **<think>** blocks emitted by DeepSeek-style models are stripped before label extraction).

G-Eval (rubric-based scoring)

For evaluations that need a custom rubric rather than a labeled pass/fail, the module supports a G-Eval-style scorer that takes a free-text rubric and returns a 0-1 score with justification.

LLM Bias metric vs. Statistical Bias Audit

VerifyWise ships two separate bias capabilities, and they are not interchangeable.

- The **LLM Bias metric** described above looks at a single response and asks a judge model whether it contains biased framing or stereotypes. This is appropriate for evaluating individual outputs during normal experiment runs.
- The **Statistical Bias Audit** module performs a quantitative fairness analysis on a CSV of decisions, applying the four-fifths rule and other group-level metrics. This is appropriate when an organization needs a regulator-ready disparate-impact report.

The two operate independently. Most organizations use the LLM Bias metric for in-experiment monitoring and the Statistical Bias Audit module separately when a formal fairness report is required.

3. How an experiment is configured

An experiment is configured in five steps inside the VerifyWise UI.

Step 1 — Project

The user selects (or creates) a project. The project's use case (chatbot, RAG or agent) determines the metric catalog available in later steps.

Step 2 — Dataset

The user selects a dataset from three sources:

- A built-in dataset shipped with the module (chatbot, RAG and agent datasets are included; multi-turn variants are also available)
- A custom dataset uploaded as a JSON file
- A set of inline prompts entered directly in the UI

The expected JSON schema is straightforward. For single-turn datasets each row contains **prompt**, optional **expected_output** and optional **context**. For multi-turn datasets each row contains a **scenario** and a **turns** array. The module auto-detects single-turn versus multi-turn by inspecting the first row.

Step 3 — Model under evaluation

The user selects the model that will produce the outputs to be scored. Supported providers include OpenAI, Anthropic, Google, Mistral, xAI, OpenRouter, Hugging Face, and any self-hosted OpenAI-compatible endpoint including Ollama. A custom endpoint URL can be provided for self-hosted or proxy deployments.

The model can optionally be linked to a model in the VerifyWise model inventory. When this link is set, the model's detail page exposes an evaluations tab that surfaces the linked experiments and bias audits alongside the model record.

Step 4 — Judge model

The user selects the LLM that will act as judge. The same provider list is supported. Common patterns are to use a high-end model from a different provider than the model under evaluation, to use a strong frontier model as judge, or to use a self-hosted judge for data residency reasons.

Step 5 — Metrics and thresholds

The user selects which metrics to run from the catalog applicable to the project's use case, and sets a numeric threshold for each. Custom scorers can be added alongside the built-in metrics in the same run.

When the experiment is submitted, the configuration is stored as a JSON snapshot on the experiment record. **This snapshot is what allows the experiment to be reproduced later.**

4. What an experiment produces

In-app results

- A summary view showing the experiment configuration, the aggregate pass rate per metric, and overall pass/fail counts
- A per-row table with the prompt, the model's response, each metric's score, the pass/fail status against the configured threshold, and the judge's reasoning where available
- Filter and sort controls for inspecting failures

Per-row visibility is important for compliance review: a reviewer can see exactly which prompts failed and exactly why the judge marked them as failing.

PDF report

Every experiment produces a formal PDF. The PDF includes:

- A cover page with project, experiment name, model under evaluation, judge model, dataset and date
- An executive summary written by an LLM, summarizing the headline findings in plain language
- A methodology section listing the metrics used and their definitions
- A per-metric results section with pass rates and example failures
- A limitations section documenting any excluded rows, missing data and known caveats
- A configuration appendix showing the full experiment configuration so the run can be reproduced

The PDF is suitable for inclusion in an audit response, a model inventory record or a vendor-risk assessment.

CSV and JSON exports

The full per-row results are available as CSV (for spreadsheet review) and JSON (for ingestion into a data warehouse or model risk management system).

LLM Arena (head-to-head comparison)

In addition to standard experiments, the module supports a head-to-head Arena mode that runs two models against the same prompts and asks a judge to compare their responses. This is useful for model selection decisions and for documenting why one model was chosen over another. Arena runs are capped at ten prompts per comparison.

5. Continuous evaluation vs. point-in-time experiments

A VerifyWise experiment is **point-in-time by design**. Each experiment is a dated record of how a specific model, against a specific dataset, with specific metrics and thresholds, behaved on a specific day.

Continuous evaluation is achieved by running experiments on a recurring cadence and retaining the results over time. There are two supported patterns:

- **CI/CD-driven evaluation.** A GitHub Action published as **verifywise/verifywise-eval-action** runs an experiment as part of a pull request or release pipeline. The action calls the VerifyWise API, polls for completion, checks the configured thresholds and posts a Markdown summary to the GitHub Actions job summary. It can gate the deployment by exiting with a non-zero status if a threshold fails. A Python SDK is also available for teams that prefer to call the API directly from their own infrastructure.
- **Manual cadence.** Organizations that prefer to keep evaluation outside the CI/CD pipeline can run experiments on a calendar cadence (weekly, monthly, quarterly) using the UI or the API. Each run produces its own PDF, and the experiments are linked in the model inventory.

Each PDF is **timestamped and retained**. When a reviewer asks how a model's behavior changed between two dates, the answer is two reports and the difference between them.

6. How findings feed the AI risk register

VerifyWise ships a separate AI risk register module. **The two modules are independent**: the experiment produces evidence, the risk register tracks the response. The handoff is currently a manual step. When an experiment shows that a metric failed its threshold, the model owner (or a designated reviewer) creates a corresponding entry in the risk register and attaches the experiment PDF as evidence.

A risk-register entry carries a description, an assigned owner, a due date, a risk classification consistent with the organization's existing risk taxonomy, and any linked controls. Remediation actions tracked there can include:

- Adjusting the prompt template
- Switching to a different model or model version
- Adding retrieval guardrails for RAG systems
- Adding a content filter for toxicity or PII
- Tightening the system prompt
- Documenting why the failure is acceptable in the production context

The two modules are **intentionally loosely coupled**. The experiment produces a dated artifact that does not depend on the rest of the platform; the risk register records the operational response. A regulator reviewing the file later sees what was measured in the PDF and what was done in the register, with the relationship between them captured by the attached evidence.

7. Mapping to NIST AI RMF, ISO/IEC 42001 and the EU AI Act

The table below shows how outputs of an LLM evaluation experiment support specific obligations.

Experiment output	NIST AI RMF function	ISO/IEC 42001 clause	EU AI Act article
Configuration snapshot per experiment	**GOVERN 1.6** — system inventory	**Clause 6.1** — planning	**Article 11** — technical documentation
Per-metric pass rates and per-row results	**MEASURE 2.3** — system performance evaluated	**Clause 9.1** — monitoring and measurement	**Article 9** — risk management
Hallucination, faithfulness, context metrics for RAG	**MEASURE 2.5** — reliability and robustness	**Clause 8.3** — operation	**Article 15** — accuracy and robustness
Toxicity, bias, conversation safety metrics	**MEASURE 2.11** — fairness and harmful bias	**Annex A.6.2** — responsible AI	**Article 10** — data and data governance
Threshold-gated CI/CD evaluation	**MANAGE 1.3** — continuous improvement	**Clause 9.1** — monitoring	**Article 72** — post-market monitoring
Limitations section and excluded rows	**MANAGE 4.3** — uncertainty communicated	**Clause 7.5** — documented information	**Article 13** — transparency
Dated PDF and JSON export retained as evidence	**GOVERN 1.2** — accountability structures	**Clause 7.5** — documented information	**Article 18** — record-keeping

8. Methodology transparency

Three points compliance teams typically raise:

Judges are LLMs, and LLMs are imperfect judges

The module is explicit about this. Every PDF documents which judge model was used, the judge prompt template (for custom scorers) and the per-row reasoning the judge gave. Reviewers can drill into per-row results to inspect pass/fail status and the judge's reasoning before accepting the experiment's findings.

Determinism

LLM-as-judge metrics are not strictly deterministic. The module supports re-running an experiment with the same configuration to measure score stability across runs. Judge temperature is configurable per experiment, with operators able to tighten it for stricter pass/fail scoring or relax it for exploratory work. For evaluations that need fully fixed scoring, custom scorers can be defined as label-based prompts where parsing extracts the verdict from a fixed set of labels.

Missing data and partial failures

Rows for which the model under evaluation failed to produce a response (timeout, rate-limit, refusal) are recorded with the failure mode and excluded from metric aggregation. The excluded counts and reasons appear in the limitations section of the PDF.

9. Common LLM evaluation use cases

The table below pairs common LLM-system patterns with the appropriate project use case, suggested metrics and a recommended cadence. It is a starting point rather than a final assignment; the right configuration depends on the organization's policies and any sector-specific requirements.

LLM system pattern	Project use case	Suggested metrics	Cadence
Customer support chatbot	Chatbot (multi-turn)	Turn relevancy, knowledge retention, conversation coherence, conversation safety, toxicity	Per release and on a calendar cadence
Internal knowledge-base assistant	RAG	Faithfulness, context relevancy, answer relevancy, correctness	Per release and on data refresh
Document Q&A on regulated content	RAG	Faithfulness, context recall, correctness, hallucination	Per release and on a calendar cadence
Agent that browses and acts	Agent	Plan quality, plan adherence, tool correctness, task completion, step efficiency	Per release and on a calendar cadence
Summarization (clinical notes, contracts, calls)	Chatbot	Correctness, completeness, faithfulness, hallucination	Per release and on a calendar cadence
Code generation assistant	Chatbot	Correctness, instruction following, custom scorer for compile/test	Per release and on every prompt change
Model selection decision	LLM Arena	Direct head-to-head comparison	At decision time and after major model releases

10. Getting started

For organizations evaluating VerifyWise's LLM Evals module, the following sequence of steps tends to work well:

1. Pick a single LLM system in production or in late development. Record it in the VerifyWise model inventory.
2. Identify a representative dataset (50 to 500 prompts) that reflects how the system is actually used.
3. Choose two to four metrics that match the system's purpose. Do not start by enabling every metric in the catalog.
4. Run a baseline experiment. Review the per-row results.
5. Adjust thresholds based on what the data shows, not on theoretical targets.
6. Schedule the experiment to re-run on each release through the CI/CD integration, or on a calendar cadence through the API.
7. Connect failed thresholds to the AI risk register so that issues are tracked alongside other AI governance work.

11. Built-in example datasets shipped with the module

The following datasets ship with VerifyWise out of the box. They are authored by the VerifyWise team and **are not** public benchmarks; **organizations are encouraged** to supplement them with internal datasets that reflect their own production traffic.

Category	Datasets
Single-turn chatbot	chatbot basic, chatbot customer support, chatbot coding helper
Multi-turn chatbot	customer support multi-turn, coding helper multi-turn, basic multi-turn
Single-turn RAG	product documentation, research papers, Wikipedia small
Multi-turn RAG	document Q&A, knowledge base, research assistant
Agent	task execution, planning, workflow automation

12. Supported providers

For both the model under evaluation and the judge model.

- OpenAI
- Anthropic
- Google (Gemini)
- Mistral
- xAI (Grok)
- OpenRouter
- Hugging Face Inference API
- Self-hosted OpenAI-compatible endpoints (including Ollama, vLLM, LM Studio and any OpenAI-API-compatible deployment)

When the VerifyWise AI Gateway is deployed in the organization's environment, custom scorer judge calls and arena model calls are routed through the gateway for spend tracking, guardrails and audit logging. Calls made by the built-in DeepEval metric judges reach providers directly.

This document describes the LLM Evals module as it ships in VerifyWise as of May 2026. Specific metrics, providers and integrations may evolve as the product develops; for the current state of any feature, contact your VerifyWise account team.